Algorithms & Data Structures

Week 06

Ahmet Feyzi Ateş

General Information

- A two phase bootcamp to integrate skills needed to think algorithmically in order to solve problems by using proper data structures
- First phase (5 weeks): Learn;
  - how to analyze a problem, break down into individual steps that can be implemented by a programming language.
  - Evaluate the time and memory costs of your implementation.
- Second phase (5 weeks): Learn;
  - How data is stored in memory
  - How different data structures make a difference while solving a problem.

General Information (2nd Phase)

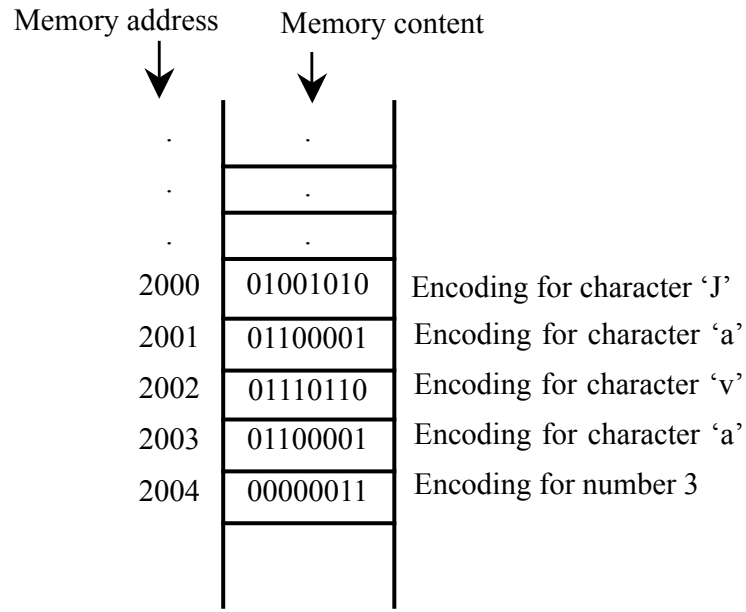
- Data, Memory, and CPU Relationship (revisited)
- Asymptotic Analysis and Big-O (revisited)
- Arrays and Dynamic Arrays (Array Lists)
- Single, Double, and Circular Linked Lists
- Stacks and Queues
- Priority Queues
- Hashing, Hash Tables, and Hash Maps
- Binary Trees and Binary Search Trees
- Minimum Heap and Maximum Heap
- Introduction to AVL and Red-Black Trees
- ***Choosing the Right Data Structure in Real-World Problems***

How Data is Stored?

- Data of various kinds, such as numbers, characters, and strings, are encoded as a series of bits (zeros and ones).

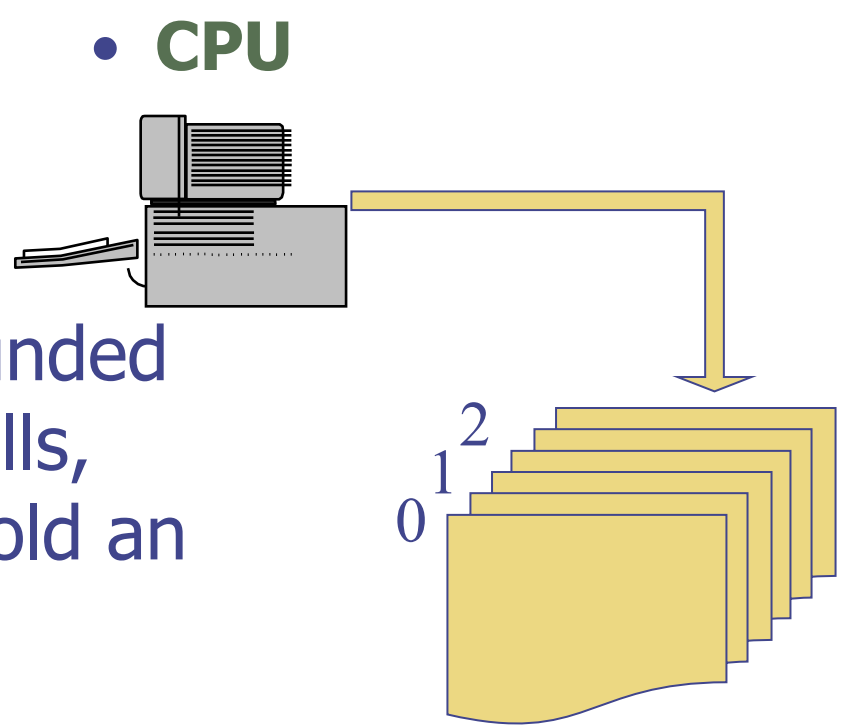
  - For example, character ‘J’ is represented by 01001010 in one byte.

- A small number such as three can be stored in a single byte. If computer needs to store a large number that cannot fit into a single byte, it uses a number of adjacent bytes.



The Random Access Machine (RAM) Model

- A **CPU**
- An potentially unbounded bank of **memory** cells, each of which can hold an arbitrary number or character
- Memory cells are numbered and accessing any cell in memory takes unit time.



Fundamental Data Structures

- lists
  - arrays, dynamic arrays
  - linked lists
  - strings
- stacks
- queues
- priority queues
- hash tables
- graphs
- trees
- sets and dictionaries

Order of growth of Running Time

- We are interested in question like:
  - *How much faster would this algorithm run on a machine that is 10 times faster than the one we have?*
  - *how much longer will the algorithm run if we double its input size?*

- ***Most important***: Order of growth within a constant multiple as $n \to \infty$
- Changing the hardware/ software environment:
  - Affects T(n) by a constant factor, but
  - Does **not** alter the growth rate of T(n)
- The **growth rate** of the running time T(n) is an intrinsic property of the algorithm itself.

"Order of Growth" Classification

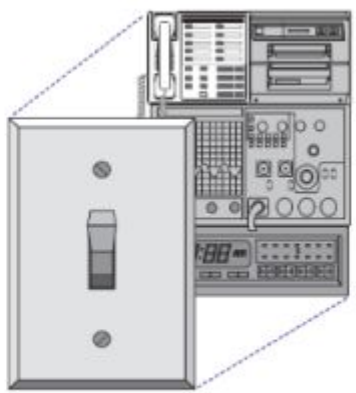
- Two functions f and g have the relation f(n) ~ g(n), *meaning that f and g grow with the same rate,* if;
  - $\lim_{n\to\infty} f(n)/g(n) = $ *<A constant value>*
- **Definition**:
  - If there is a positive constant so that the following equation holds:
    - $f(n) \sim c * g(n)$
  - Then the order of growth of the two functions ***f and g are the same.***
    - or, in other words:
  - The order of growth classification of ***f(n) is g(n),***
    - *or shortly,* ***f(n) is O(g(n))***
- "Order of growth" definition ignores leading coefficients and lower order terms in a function.

Design Principles

Chief among the principles of the software design process are the following:



Abstraction



Encapsulation



Modularity

Abstraction

The notion of abstraction is to distill a complicated system down to its most fundamental parts and describe these parts in a simple, precise language.

Typically, describing the parts of a system involves naming them and explaining their functionality.

Applying the abstraction paradigm to the design of data structures gives rise to ***abstract data types (ADTs)***.

An ADT is a mathematical model of a data structure that specifies the type of the data stored, the operations supported on them, and the types of the parameters of the operations.

An ADT specifies **what** each operation does, but **not how** it does.

Abstract Data Type (ADT) in one slide



ADT: [Name]

1. Data (What it holds): A conceptual description of the values and their organization (e.g., *"An ordered collection of elements where items are added and removed from the front"*).

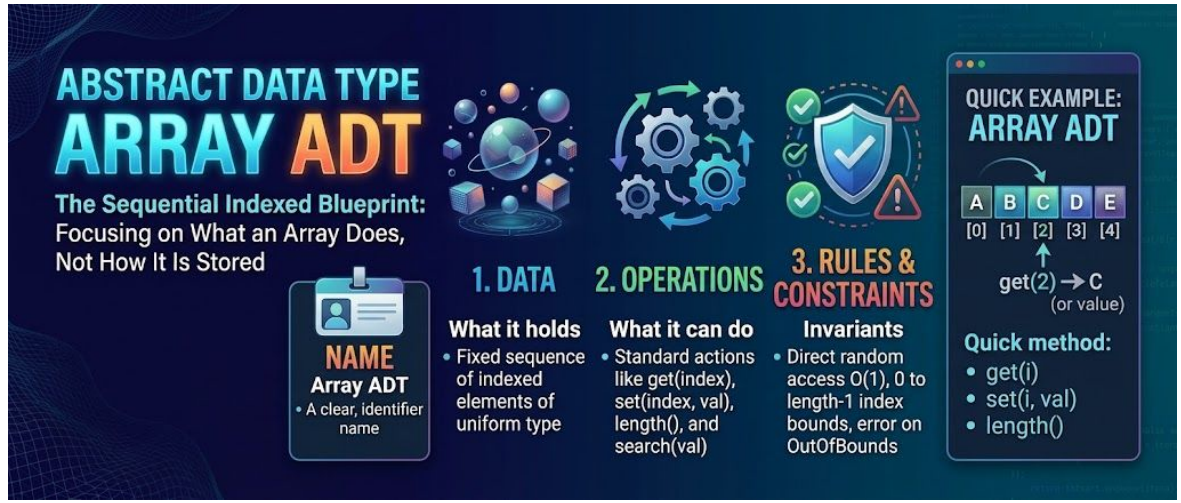
2. Operations (What it can do):

- init() → Creates an empty instance.
- add(element) → Adds an element to the collection.
- remove() →Removes and/or returns an element.
- get() → Inspects an element without removing it.
- isEmpty() → Returns true if empty, false otherwise.
- size() → Returns the number of items.

3. Rules & Constraints:

- **Order:** First-In First-Out (FIFO), Last-In First-Out (LIFO), keyed, or unordered.
- **Errors:** What happens on an illegal action (e.g., calling remove() when empty).

Arrays



ADT: Array ADT

1. Data : A finite, ordered sequence of homogeneous elements where each item is uniquely mapped to a contiguous numerical index from 0 to length - 1.

2. Operations:

- init(capacity) ⟶ Creates an array with a fixed number of slots.
- get(index) ⟶ Retrieves and returns the element at the specified position.
- set(index, value) ⟶ Replaces or assigns the element at the specified position.
- length() ⟶ Returns the total capacity or count of slots (i.e., size: how many data elements the array holds)..
- search(value) ⟶Finds and returns the index of the first occurrence of value (or indicates absence).

3. Rules & Constraints:

- **Direct Random Access:** Any element can be read or written directly using its index in O(1) constant time.
- **Bounds Enforcement:** Valid indices satisfy 0 ≤ \text{index} < length. Accessing an index outside this range raises an IndexOutOfBounds error.
- **Fixed Structure:** In classical fixed-capacity arrays, the size is fixed upon creation and cannot dynamically expand without allocating a new array.

Arrays

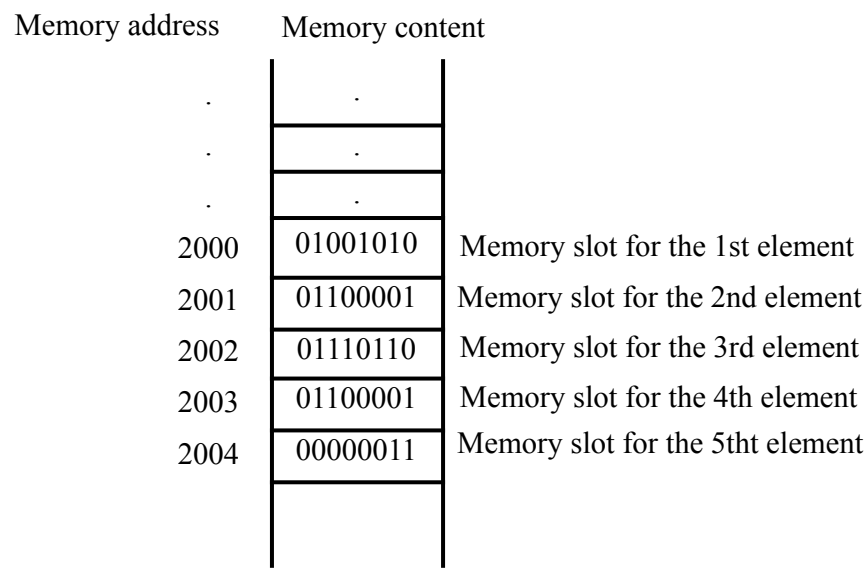


- An Array ADT is a finite, ordered collection of elements of the same type, where each element is identified by an integer index.
  - if we anticipate need for expansion in an array's size in the future, we must first declare the array with the **maximum** possible size in the first place.
- Arrays are arguably one of the simplest data structures that we are going to study.
- Nevertheless, they form the basis of techniques that are used repeatedly to build more sophisticated data structures.
- These other structures may be more general than a simple array-based solution, and they may support many more operations.
- But studying the concrete array data structure is a great starting point for understanding these more sophisticated structures, since every data structure has to be implemented using concrete means.

Arrays

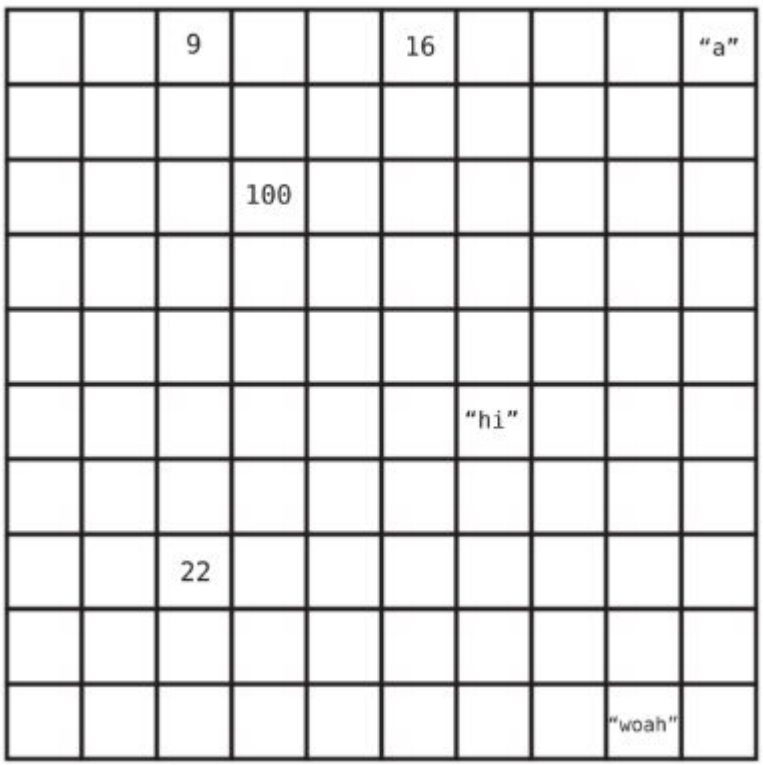


- An array is a collection of elements of the same type.
  - *Given any type T and a constant N, a variable of type T[n] holds an array of n elements, each of type T.*
  - *Each element of the array is referenced by its index, that is, a number from 0 to n − 1.*
  - *Generally, array elements are stored in consecutive locations in memory.*
  - *Once declared, all basic operations (reading from a given index position, and writing to a given index position) on arrays can be performed in constant time, i.e., O(1).*
  - *Once declared, it is **not** possible to increase the number of elements in an array.*

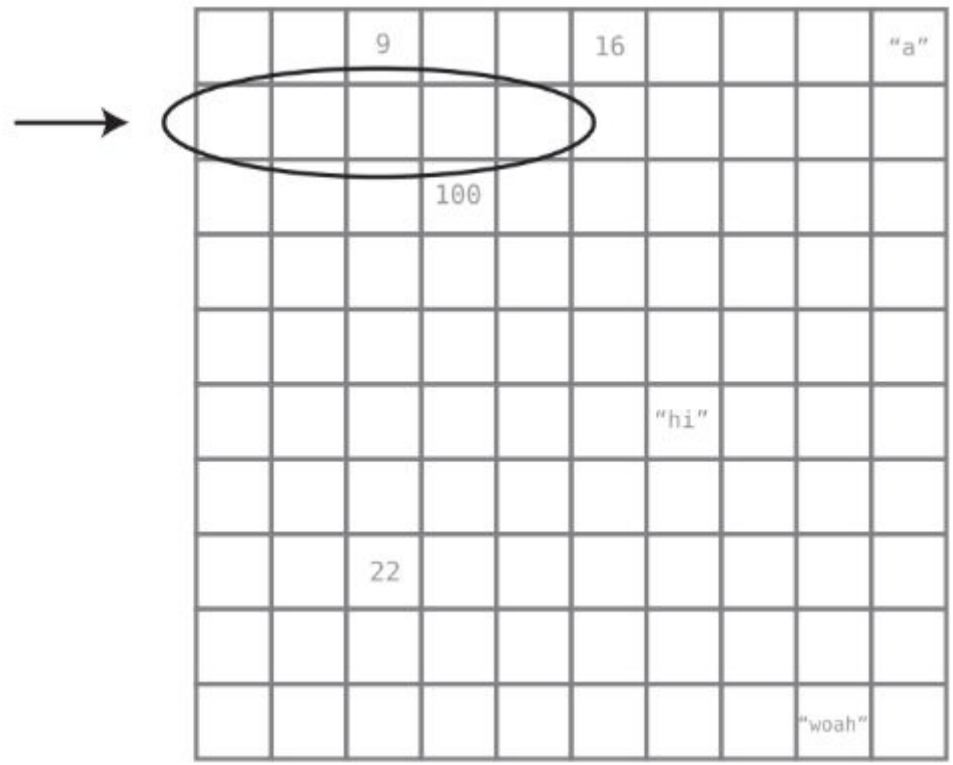


Arrays

A computer's memory can be viewed as a giant collection of cells. In the following diagram, you can see a grid of cells in which some are empty and some contain bits of data:



When a program declares an array, it allocates a contiguous set of empty cells for use in the program.
So, if you were creating an array meant to hold five elements, your computer would find a group of five empty cells in a row and designate it to serve as your array:



Arrays

Now, every cell in a computer's memory has a specific address.
Each cell's memory address is one number greater than the previous cell's address.

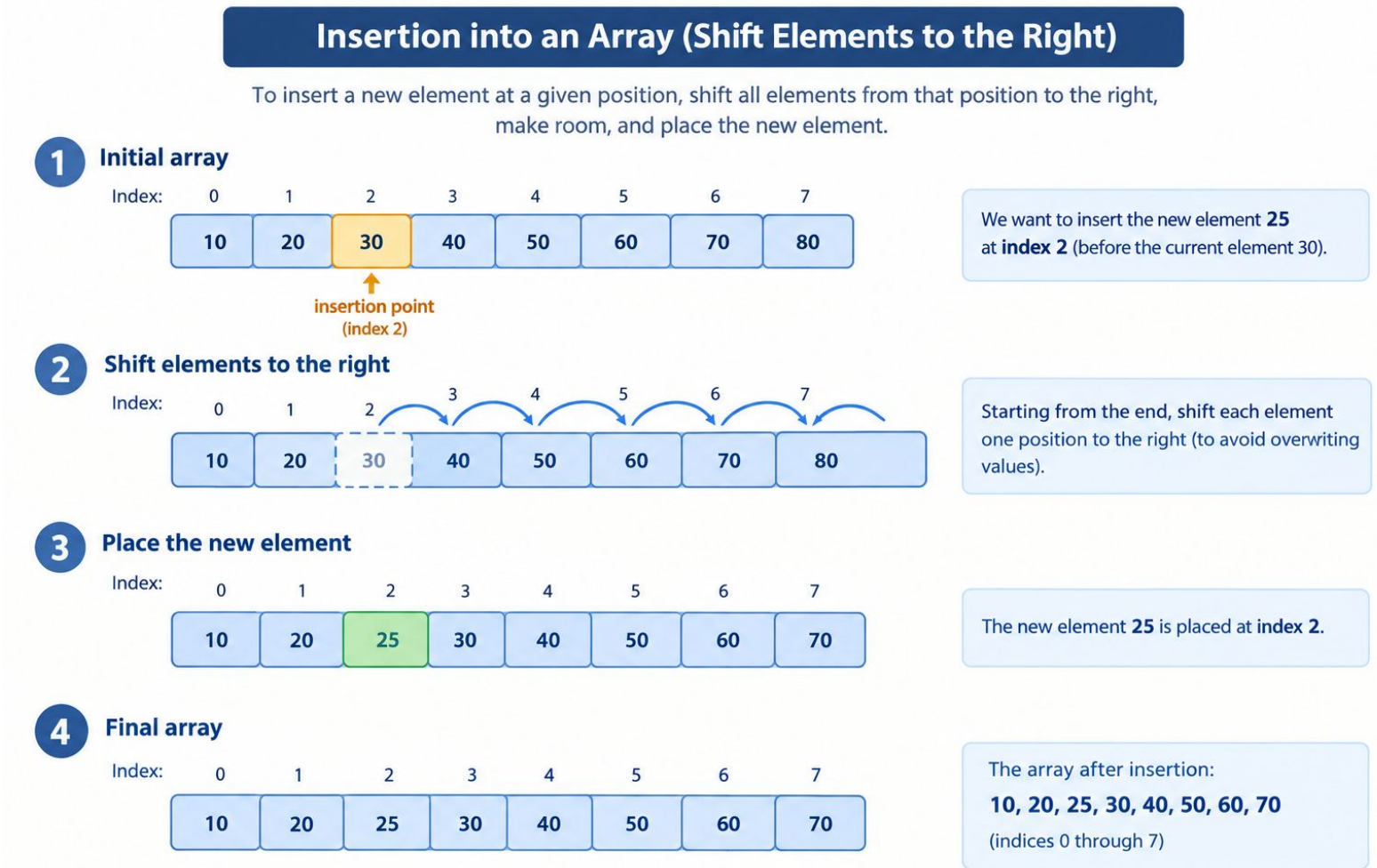
1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
1010	1011	1012	1013	1014	1015	1016	1017	1018	1019
1020	1021	1022	1023	1024	1025	1026	1027	1028	1029
1030	1031	1032	1033	1034	1035	1036	1037	1038	1039
1040	1041	1042	1043	1044	1045	1046	1047	1048	1049
1050	1051	1052	1053	1054	1055	1056	1057	1058	1059
1060	1061	1062	1063	1064	1065	1066	1067	1068	1069
1070	1071	1072	1073	1074	1075	1076	1077	1078	1079
1080	1081	1082	1083	1084	1085	1086	1087	1088	1089
1090	1091	1092	1093	1094	1095	1096	1097	1098	1099

When the computer reads a value at a particular index of an array, it can jump straight to that index because of the combination of the following facts about computers (Remember our RAM Model):

1. A computer can jump to any memory address in one step.
2. Whenever a computer allocates an array, it also makes note at which memory address the array begins.
3. However, a computer can also find the value at any index by performing simple addition.

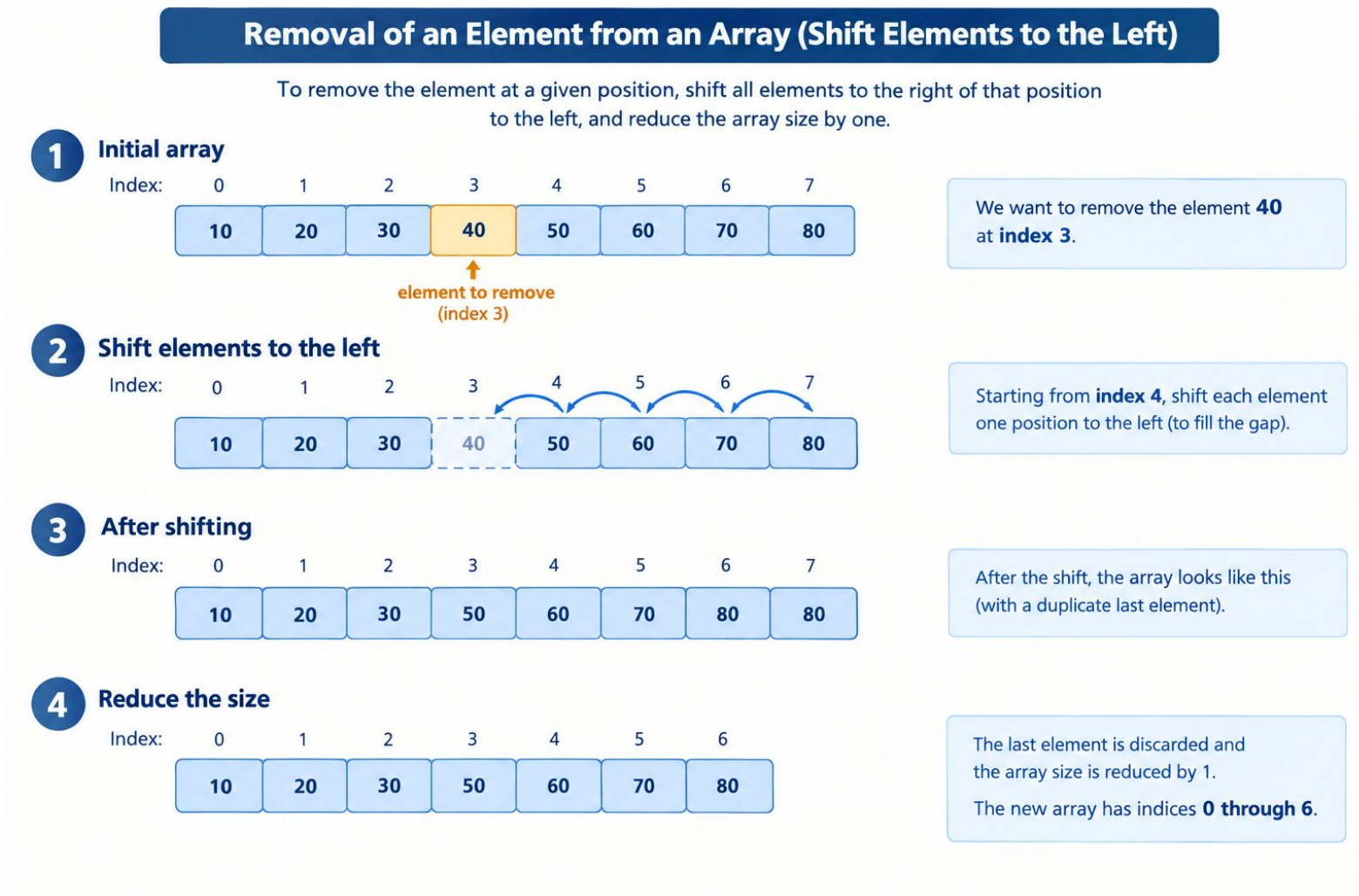
Element Insertion in Arrays

- *insert(i)*: The approach is to shift all the entries of the array after the ith index to the right, in order to make space for the new entry.

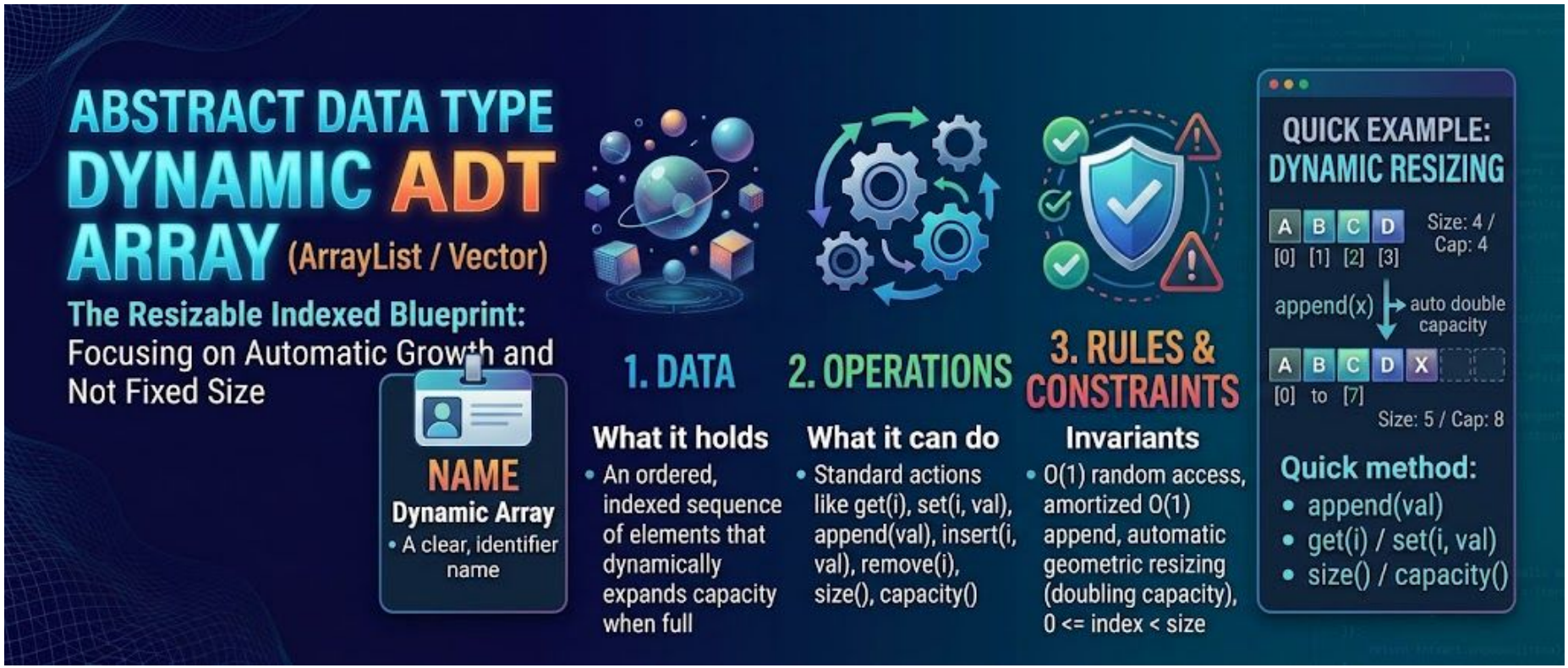


Element Deletion (Removal) in Arrays

- *delete(i)*: Removes and returns the game entry e at index i in the array. all objects previously stored at indices higher than i are "shifted left" to fill in for the removed.



Dynamic Arrays



Dynamic Arrays

ADT: Dynamic Array ADT

1. Data : A dynamically resizable, ordered sequence of elements where each item is mapped to a zero-based index 0 ≤ index} < size. It maintains two distinct metrics:

- **Size:** Number of elements currently stored.
- **Capacity:** Number of available slots reserved in underlying memory.

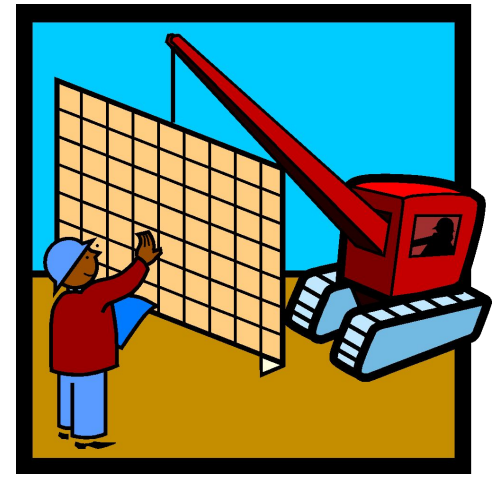
2. Operations:

- init(initialCapacity) ⟶ Creates an empty dynamic array with a default or specified capacity.
- get(index) ⟶ Returns the element at the specified index (O(1)).
- set(index, value) ⟶ Overwrites the element at the specified index (O(1)).
- append(value) / add(value) ⟶ Appends an element to the end; automatically doubles capacity when size = capacity (amortized O(1)).
- insert(index, value) ⟶ Shifts subsequent elements right and inserts at position (O(n)).
- remove(index) ⟶ Shifts subsequent elements left and returns the removed item (O(n)).
- size() ⟶Returns the count of active elements (O(1)).
- isEmpty() ⟶Checks if size = 0

3. Rules & Constraints:

- **Direct Random Access:** Reading or modifying any occupied position runs in O(1) time.
- **Automatic Geometric Growth:** When full, the container allocates a larger memory block (typically 1.5 to 2 times the old capacity, copies elements across, and reclaims old memory to achieve an amortized O(1) append time.
- **Bounds Enforcement:** Valid indices are strictly bounded by 0≤index<size (not capacity). Accessing an index outside this range triggers an out-of-bounds error.

The Array List ADT as Dynamic Array



- The Vector or Array List ADT extends the notion of array by storing a sequence of objects
- An element can be accessed, inserted or removed by specifying its index (number of elements preceding it)
- An exception is thrown if an incorrect index is given (e.g., a negative index)

Main methods:

- at(integer i): returns the element at index i without removing it
- set(integer i, object o): replace the element at index i with o
- insert(integer i, object o): insert a new element o to have index i
- erase(integer i): removes element at index i

Additional methods:

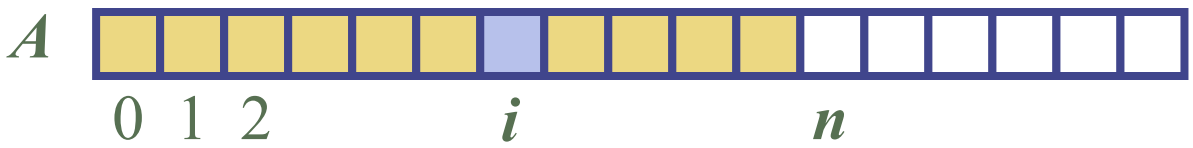
- size()
- empty()

Applications of Array Lists

- Direct applications
  - Sorted collection of objects (elementary database)
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

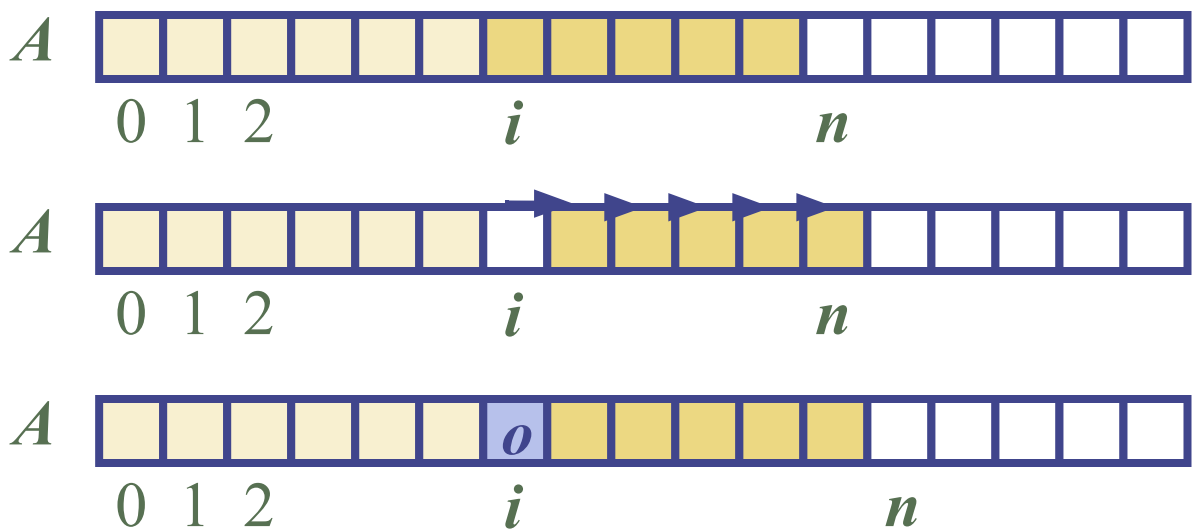
Array-based Implementation

- Use an array A of size N
- A variable n keeps track of the size of the array list (number of elements stored)
- Operation $at(i)$ is implemented in $O(1)$ time by returning $A[i]$
- Operation $set(i,o)$ is implemented in $O(1)$ time by performing $A[i] = o$



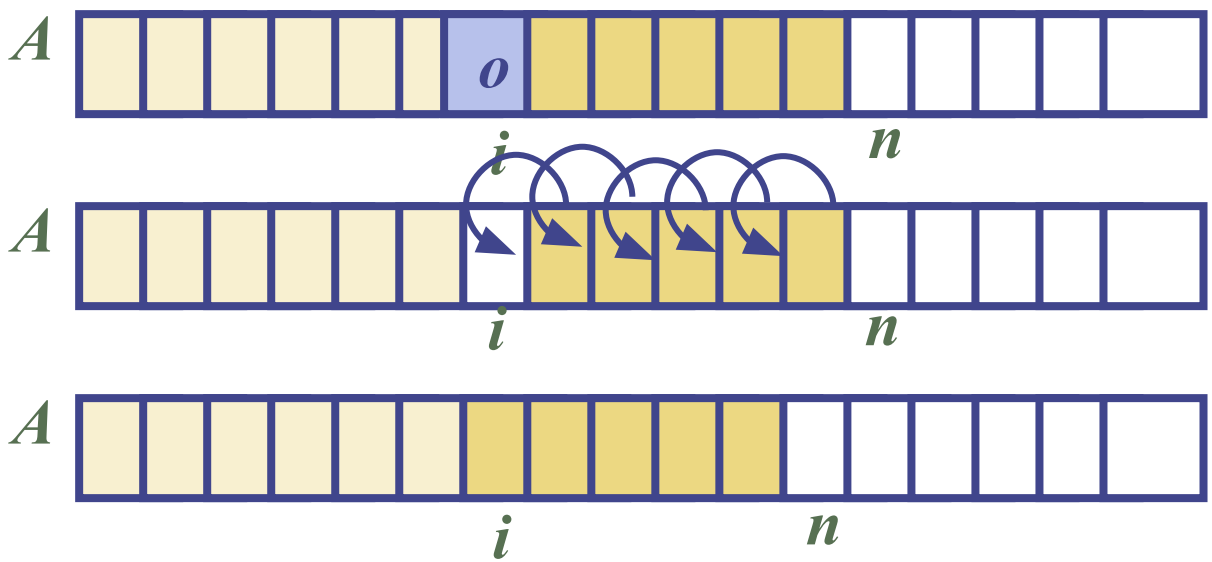
Insertion

- In operation ***insert***(i, o), we need to make room for the new element by shifting forward the $n - i$ elements $A[i], \ldots, A[n-1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



Element Removal

- In operation *erase*(i), we need to fill the hole left by the removed element by shifting backward the $n - i - 1$ elements $A[i + 1], \ldots, A[n - 1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



Performance

- In the array based implementation of an array list:
  - The space used by the data structure is $O(n)$
  - ***size***, ***empty***, ***at*** and ***set*** run in $O(1)$ time
  - ***insert*** and ***erase*** run in $O(n)$ time in worst case
- If we use the array in a circular fashion, operations ***insert***$(0, x)$ and ***erase***$(0, x)$ run in $O(1)$ time
- In an ***insert*** operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one

Growable Array-based Array List

- In an insert(o) operation (without an index), we always insert at the end
- When the array is full, we replace the array with a larger one
- How large should the new array be?
  - Incremental strategy: increase the size by a constant c
  - Doubling strategy: double the size

```
Algorithm insert(o)
    if t = S.length − 1 then
        A ← new array of
                size …
        for i ← 0 to n−1 do
            A[i] ← S[i]
        S ← A
    n ← n + 1
    S[n−1] ← o
```

Comparison of the Strategies

- We compare the incremental strategy and the doubling strategy by analyzing the total time $T(n)$ needed to perform a series of n insert(o) operations.
- We assume that we start with an empty array represented by an array of size 1
- We call amortized time of an insert operation the average time taken by an insert over the series of operations, i.e., $T(n)/n$

Incremental Strategy Analysis

- We replace the array $k = n/c$ times
- The total time $T(n)$ of a series of n insert operations is proportional to

$$n + (n+c) + (n+2c) + (n+3c) + (n+4c) + \ldots + (n+kc) =$$

$$n + c(1 + 2 + 3 + \ldots + k) =$$

$$n(k+1) + ck(k + 1)/2 =$$

$$nk + n + ck^2/2 + ck/ =$$

- Since $k = n/c$, replacing k with n/c *yields:*

$$n^2/2 + n + cn^2/4$$

- Since c is a constant, $T(n)$ is $O(n^2)$
- The amortized time of an insert operation is $O(n)$

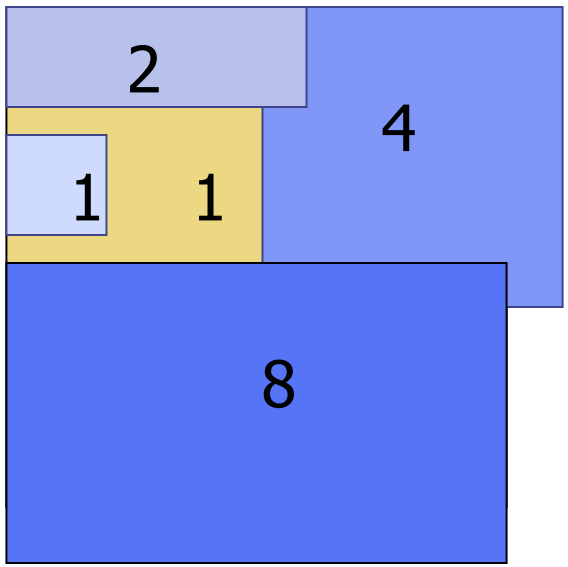
Doubling Strategy Analysis

- We replace the array $k = \log_2 n$ times
- The total time $T(n)$ of a series of n insert operations is proportional to

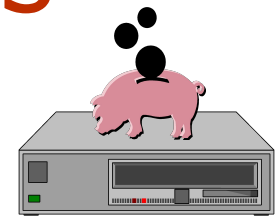
$$n + 1 + 2 + 4 + 8 + \ldots + 2^k =$$
$$n + 2^{k+1} - 1 =$$
$$3n - 1$$

- $T(n)$ is $O(n)$
- The amortized time of an insert operation is $O(1)$

geometric series



Accounting Method Analysis of the Doubling Strategy

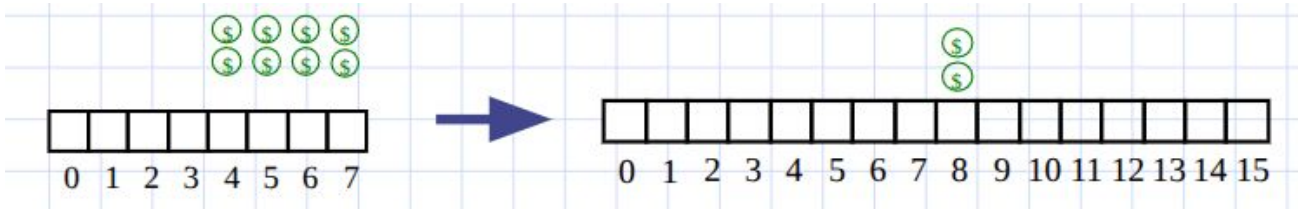


- The **accounting method** determines the amortized running time with a system of credits and debits
- We view a computer as a coin-operated device requiring 1 cyber-dollar for a constant amount of computing.
  - We set up a scheme for charging operations. This is known as an **amortization scheme**.
  - The scheme must give us always enough money to pay for the actual cost of the operation.
  - The total cost of the series of operations is no more than the total amount charged.
- (amortized time) ≤ (total $ charged) / (# operations)

Amortization Scheme for the Doubling Strategy



- Consider again the *k* phases, where each phase consisting of twice as many pushes as the one before.
- At the end of a phase we must have saved enough to pay for the array-growing push of the next phase.
- At the end of phase *i* we want to have saved *i* cyber-dollars, to pay for the array growth for the beginning of the next phase.



- We charge $3 for a push. The $2 saved for a regular push are "stored" in the second half of the array. Thus, we will have $2(i/2)=i$ cyber-dollars saved at then end of phase *i*.
- Therefore, each insert runs in $O(1)$ amortized time; n inserts run in $O(n)$ time.

ArrayList Animations

https://vizlearn.app/algorithms/data-structures/array-list

Sample Java Applications

Normal (Standard) Array

https://github.com/ahmetfeyziates/Algorithms-and-DataStructures-Bootcamp/blob/main/src/ArrayExample.java

Dynamic Array (ArrayList)

https://github.com/ahmetfeyziates/Algorithms-and-DataStructures-Bootcamp/blob/main/src/ArrayListTutorial.java